

Lecture 2

The Hopfield Network Pt.1

Elena Raponi | Neural Computing course



Universiteit
Leiden
The Netherlands

Discover the world at Leiden University

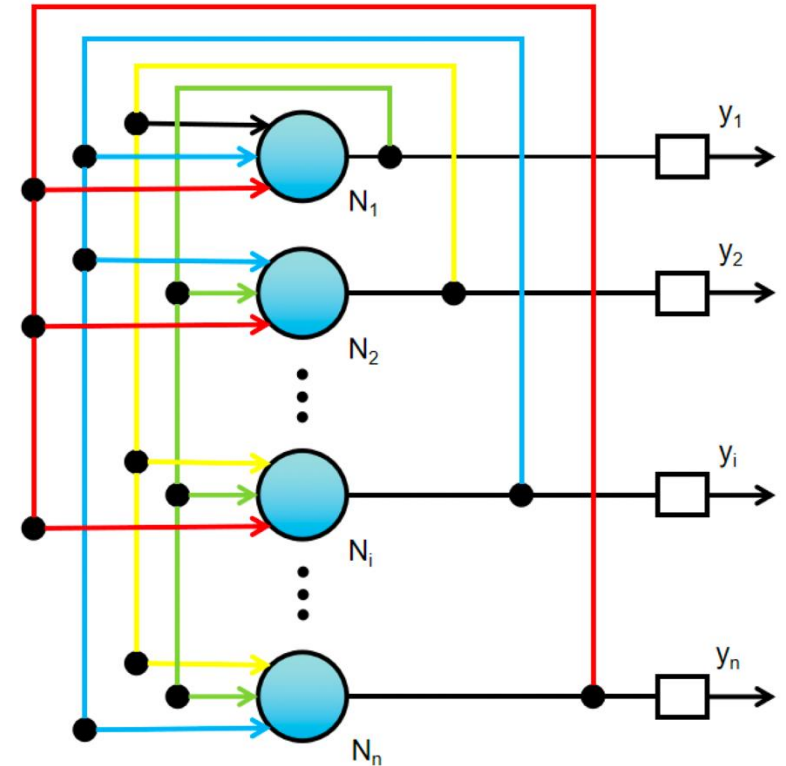
Index

1. Associative Memory
2. The Hopfield Network
3. Different visualizations
4. Formal Definition
5. Stability Condition and Attractors
6. Energy Function



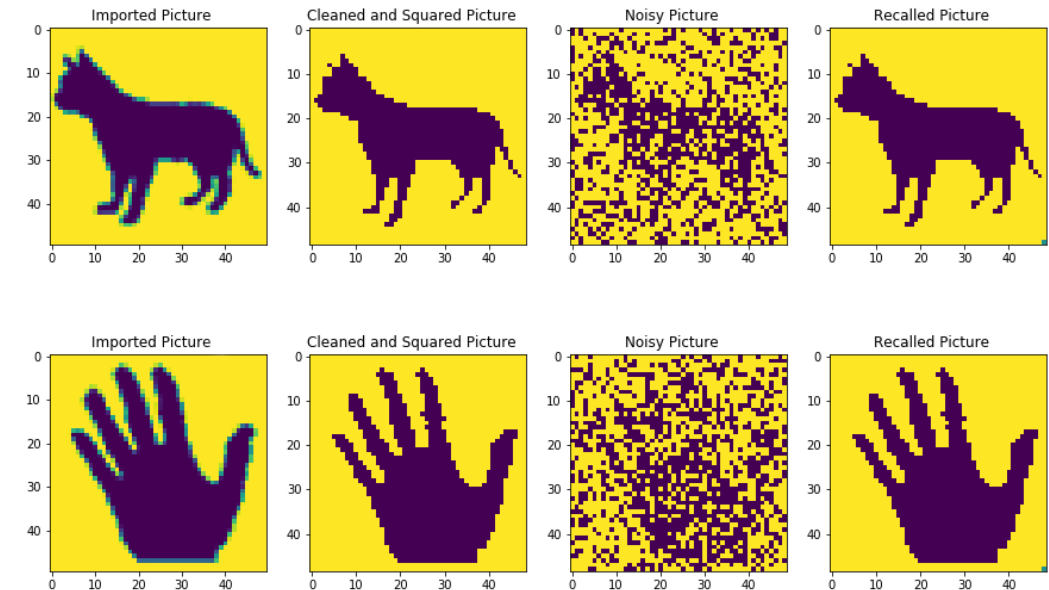
Why the Hopfield Network?

1. A **Hopfield Network (HN)** is a recurrent neural network capable of storing and retrieving multiple memories
2. This simple machine is both an informative and aesthetically pleasing point of entry into the study and modeling of memory and information processing in neural networks
3. It addresses the **Associative Memory Problem**



Associative memory

- Associative memory is also known as content-addressable memory
- It is a system where recalling a memory is triggered by the **similarity between an input pattern and a stored one**.
- Deals with **incomplete or noisy information**.



An image can be viewed as a grid of pixel values. If the image has pixels that are either black or white (binary states), then a binary Hopfield network (HN) can accurately represent it using these binary states.

The Hopfield Network

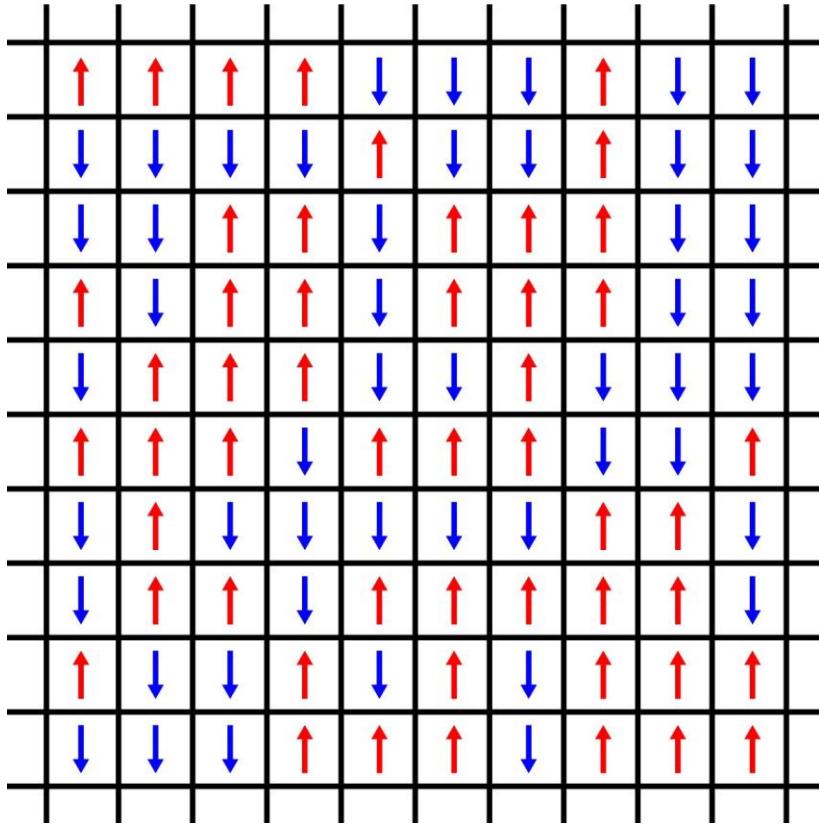


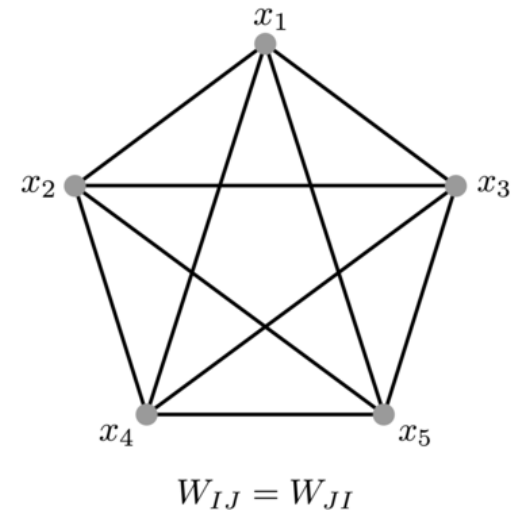
Figure: The Ising model.

- A HN is neural network represented by a **2D graph** and each unit occupies one of **two states**: active or inactive
- It is **inspired by the Ising model**, a physics model that explains the emergent behavior of the magnetic fields produced by ferromagnetic materials
- Aligned with this, the dynamic behavior of a HN is ultimately determined according to a metaphorical notion of **‘system energy’**
- The network converges to a state of **‘energetic minimum’**, which in the case of a learned network happens to be a **memory**, also referred to as **stored pattern**

The Hopfield Network

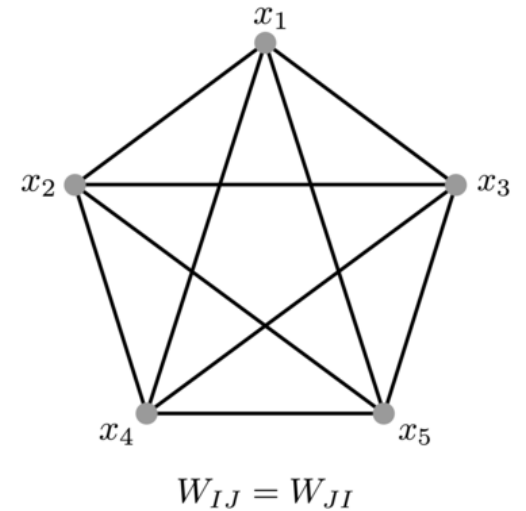
In a HP, each unit within the network possesses three primary characteristics:

1. **Connectivity:** Every neuron in the network is linked to all other neurons, and each connection carries a distinct weight, analogous to the strength of a synapse. These weights, representing connection strengths, are stored in a weight matrix.
2. **Activation:** This is determined by the net input received from other neurons and their corresponding connection weights, resembling the membrane potential of a neuron.
3. **Bipolar state:** This represents the output of the neuron, calculated using its activation and a thresholding function, analogous to a neuron's firing state. In this scenario, the output values are restricted to -1 and +1.



The Hopfield Network

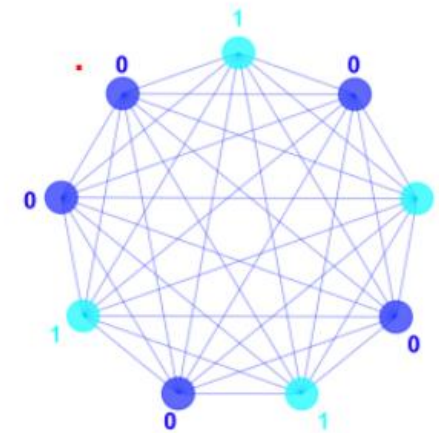
- **Connections** between neurons in a HN are **symmetric**. This means that if neuron i is connected to neuron j with a strength of $+1$, neuron j 's connection to neuron i is also $+1$.
- The ***weights*** of those connections are the **tunable** parameter.
- When we speak of ‘network learning’ what we really mean is the fine-tuning of each weight in the network to achieve the retrieval of a stored pattern.



Different visualizations

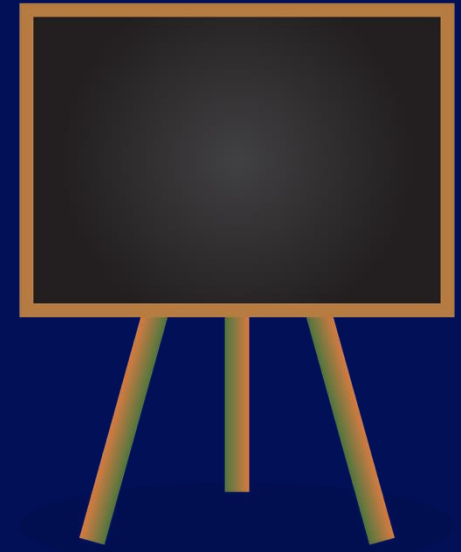
- In a computer, the state of each neuron is represented as the individual elements in a $1 \times N$ vector.
- The vector can also be rearranged to form a matrix (this recalls the pixels in an image)
- Another way to visualize this is to represent the network as a graph

Example: $\mathbf{S} = [0,1,0,1,0,1,0,1,0]$



Let's formalize on the blackboard

(a recap on the slides will follow)



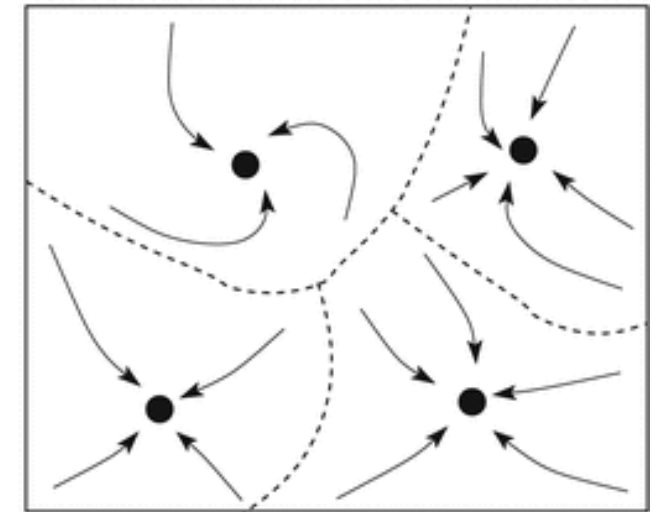
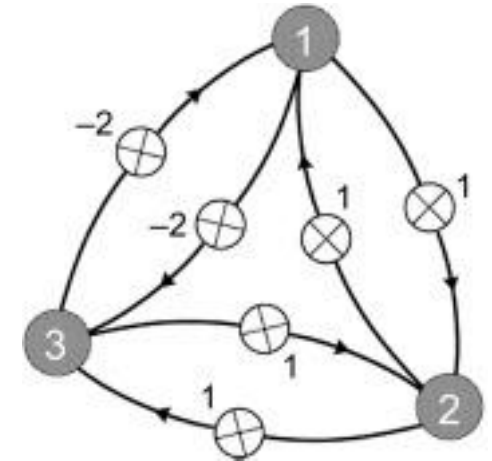
Formal Definition

- General model of the HN:

$$O_i(t + 1) = g \left(\sum_j w_{ij} O_j(t) + b_i \right)$$

where the activation function $g(x)$ can be tanh or sigmoid

- Associative memory
 - Capable of store a collection of patterns $\{\vec{\xi}_i\}_{1 \leq i \leq p}$
 - If present a new (not stored) pattern $\vec{x}$, the network will produce the stored pattern most closely resemble $\vec{x}$.
- Given an input, the model will evolve into a **stable state**
- The stable states partition the input into attracting basins/regions.



Stability Condition

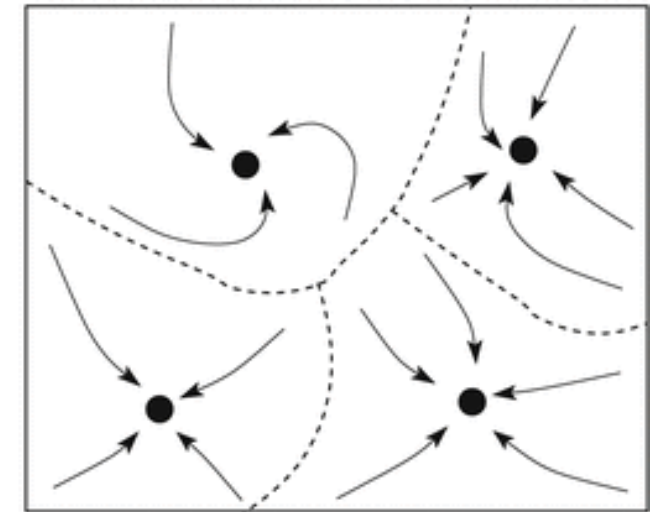
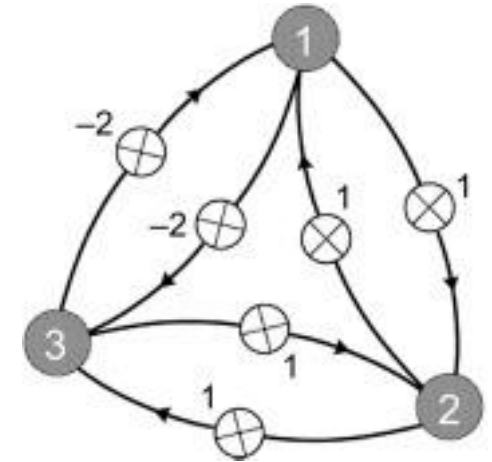
- Let's consider a simplified model of the HN:

$$O_i(t + 1) = \text{sgn} \left(\sum_j w_{ij} O_j(t) \right)$$

and a stored pattern $\vec{\xi}$. The condition for it to be a stable pattern is:

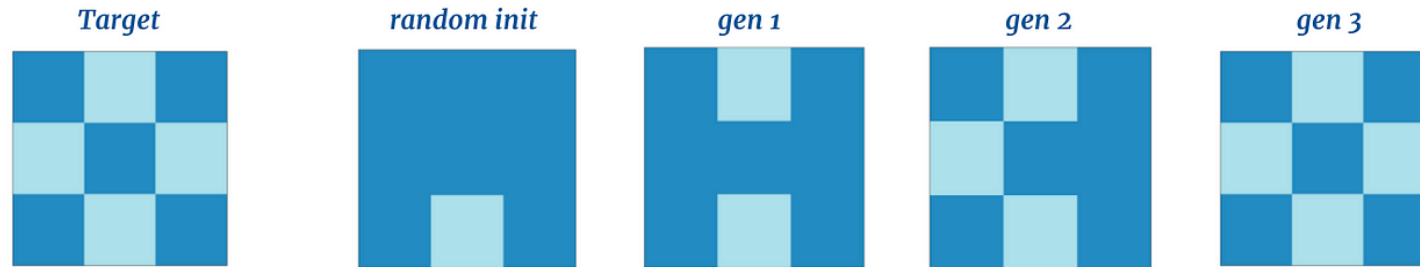
$$O_i = \text{sgn} \left(\sum_j w_{ij} \xi_j \right) = \xi_i$$

- If we set the weights to $w_{ij} \propto \xi_i \xi_j$, we will get $\vec{\xi}$ back from the network



Attractors

Let's initialize the network to a random input pattern $\vec{x}$, and let the network run according to the update rules described above, one neuron at a time:



- If we set the weights to $w_{ij} \propto \xi_i \xi_j$, we will get $\vec{\xi}$ (or the reversed pattern) back from the network.
- $\vec{\xi}$ is an **ATTRACTOR** for the **HN**.
- What if more than half of the bits in $\vec{x}$ is wrong? Stay tuned!

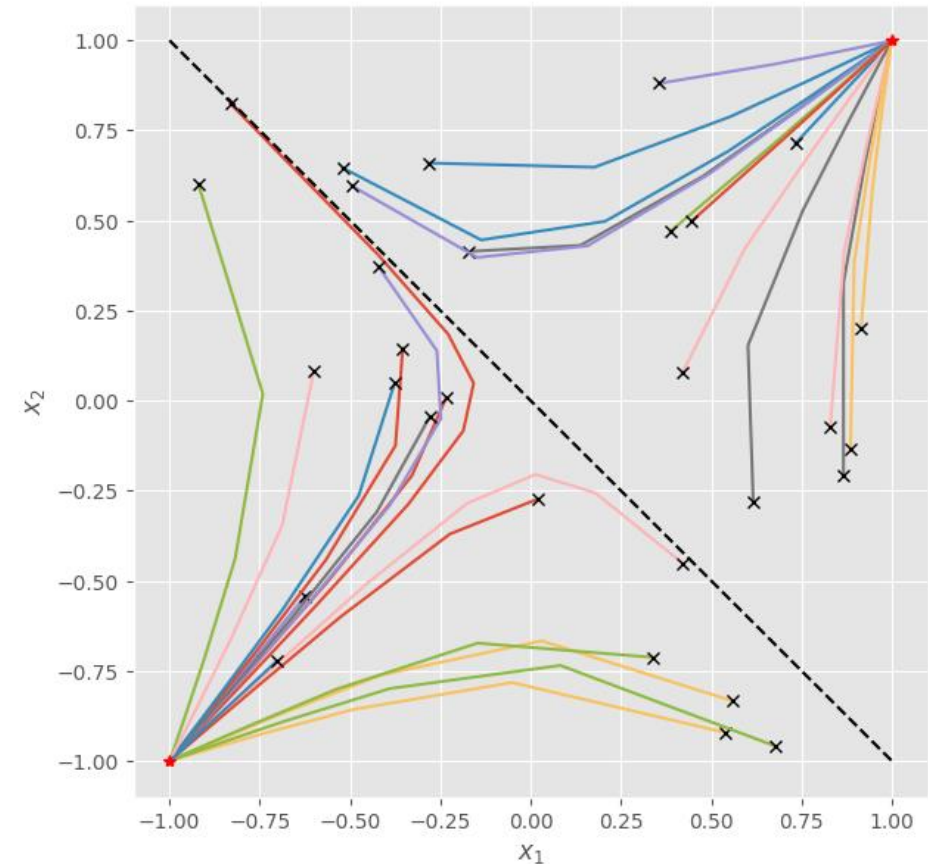
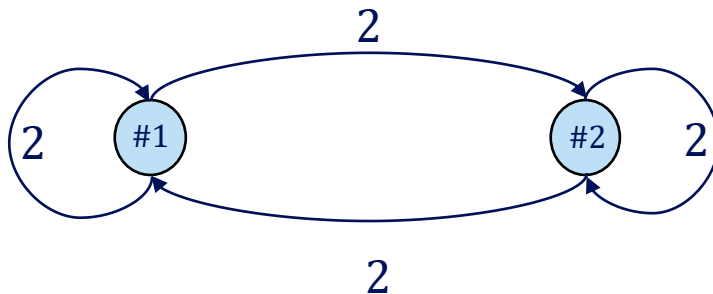
Example of a HN

An example:

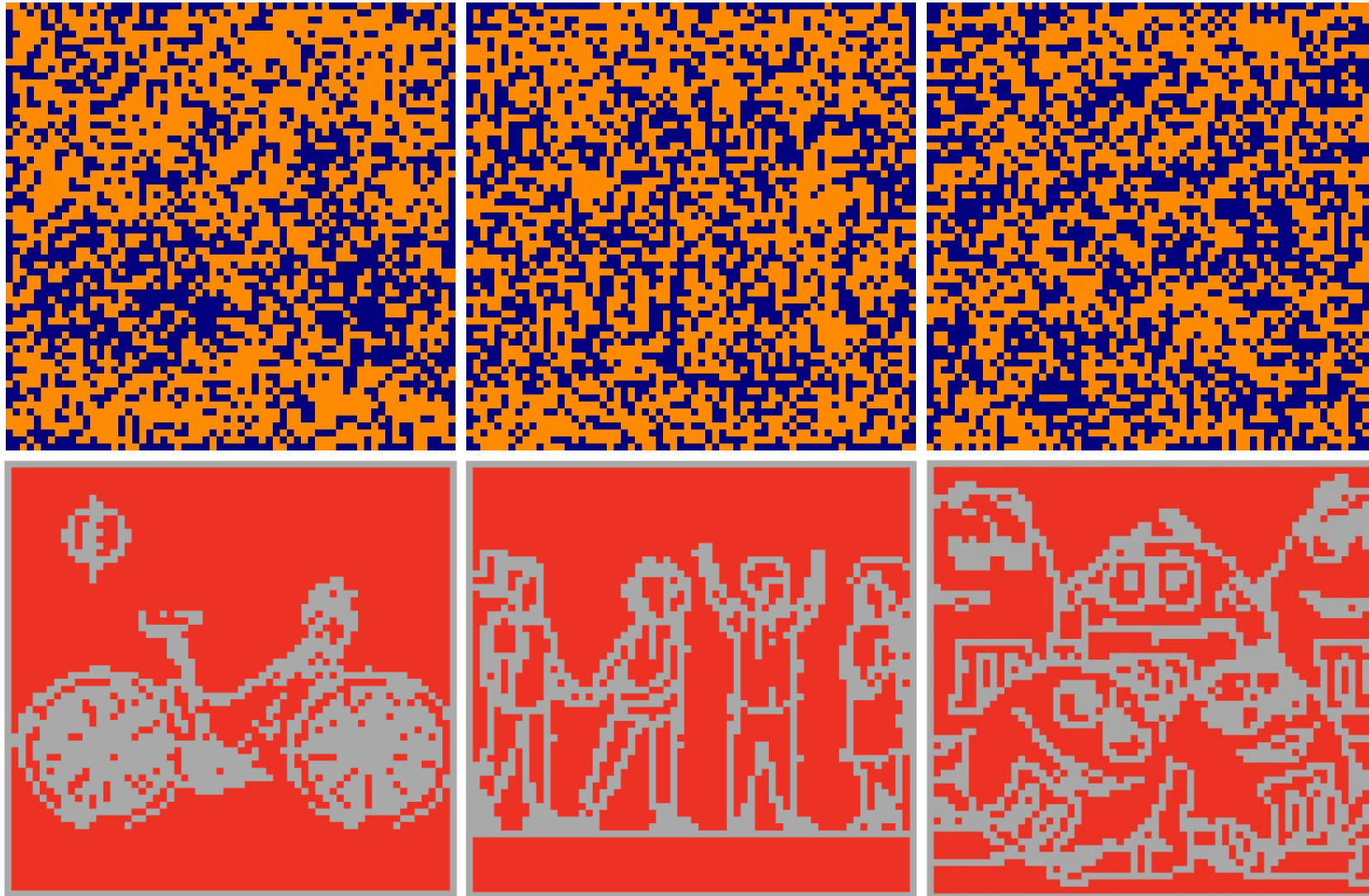
$$O_1(t+1) = \tanh(2 \cdot O_1(t) + 2 \cdot O_2(t))$$

$$O_2(t+1) = \tanh(2 \cdot O_1(t) + 2 \cdot O_2(t))$$

- Two inputs in the range $[-1, 1]$
- tanh activation; no bias term
- Evolve for 10 steps
- The model:

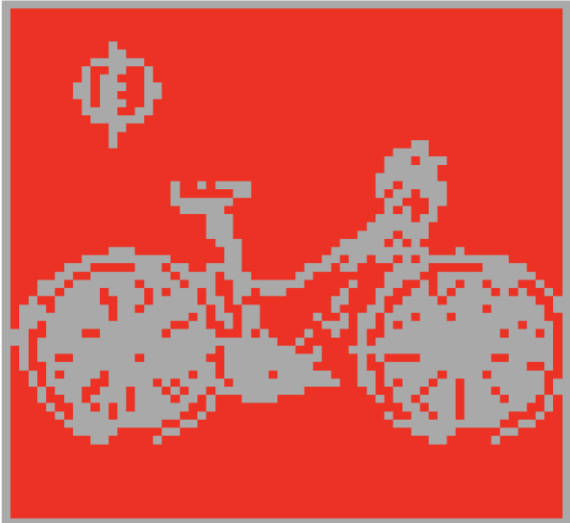
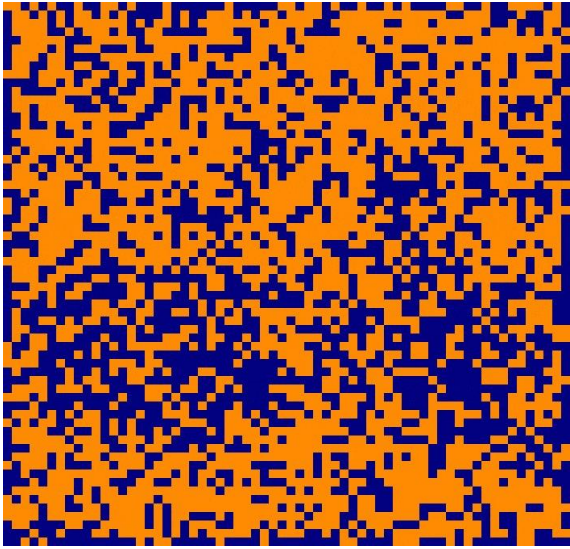


Example of multiple stored patterns



[Source: <https://towardsdatascience.com/hopfield-networks-neural-memory-machines-4c94be821073>]

Example of multiple stored patterns



- In this example above, a HN of 4096 neurons has memorized three distinct patterns (the red images)
- Each pattern is a 64x64 pixel image
- The network is initialized 3 times from a distorted version of each image
- The network restores each respective pattern as its state converges
- **CAPACITY:** Networks configured in this way are actually capable of storing up to $\sim 0.15N$ patterns, where N is the number of neurons in the network (we will see this in more detail)

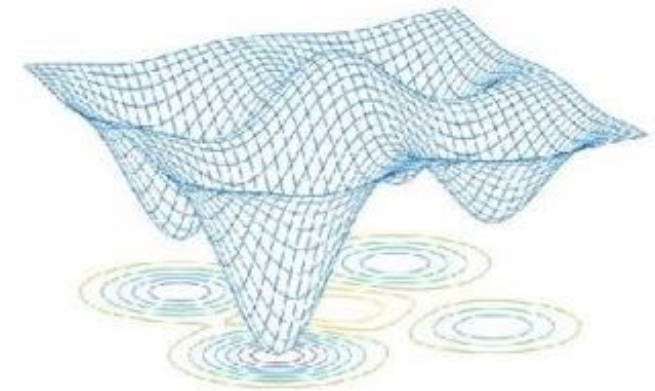
Energy Function

- The model:

$$O_i(t + 1) = g \left(\sum_j w_{ij} O_j(t) + b_i \right)$$

- **Stable states** are the minimizer of the energy function:
- The **Energy function** monotonically decreases with each subsequent update generation
- We could start from this formula and use automatic gradient descent algorithm to find the weights!

$$E(\vec{x}) = -\frac{1}{2} \vec{x}^\top \mathbf{W} \vec{x} - \vec{x}^\top \vec{b}$$





Thank you – Any questions?



Universiteit
Leiden
The Netherlands

e.raponi@liacs.leidenuniv.nl

Discover the world at Leiden University